System Testing

System testing is the final testing phase in which the complete and integrated software system is evaluated to verify that it satisfies all specified requirements. Unlike unit or integration testing, system testing focuses on validating the system as a whole, including the interaction between models, controllers, views, and external components.

The purpose of system testing in this project is to ensure that all functional requirements related to Cashiers, Managers, and Administrators behave correctly from an end-user perspective. Tests are designed to simulate real system usage by invoking controller methods and, where possible, interacting with JavaFX views. Each requirement is validated against its expected behavior, and any limitations encountered during testing are documented and justified.

REQ-09

The system shall automatically update item stock levels after each sale.

Before executing this test, all required objects were initialized in the `@BeforeEach` setup method in order to avoid redundancy and unnecessary resource consumption.
To test this requirement, we simulated a sale by reducing the quantity of an item through the ManagerController, which represents the system logic responsible for stock management. The test verified that after a decrease in stock, the item quantity was updated correctly and that the item was marked as out of stock when its quantity reached zero.
The test was successful, confirming that the system automatically updates stock levels after each sale.

REQ-10

The system shall allow Managers to add new items to the inventory.

This requirement was tested using the ManagerController, which is the system entry point for inventory management actions performed by Managers. A new item was created and

added through the controller, after which the inventory list was checked to ensure that the item was correctly inserted.

The test was successful, proving that Managers are able to add new items to the inventory through the system.

REQ-11

The system shall allow Managers to update or remove existing inventory items.

To test this requirement, an existing item's stock was increased using the ManagerController and its updated quantity was verified. Subsequently, the same item was removed from the inventory using its name as an identifier.

The test was successful, confirming that the system allows Managers to both update and remove inventory items.

REQ-12

The system shall allow Managers to organize items by category.

This requirement was tested by adding a new item to a specific sector, which represents an item category in the system. The test verified that the item was correctly stored within the corresponding sector and that the sector's item count was updated accordingly.

The test was successful, confirming that the system allows Managers to organize items by category.

REQ-13

The system shall notify Managers when an item's stock falls below a predefined threshold.

To test this requirement, the quantity of an item was manually set below the predefined minimum threshold. The ManagerController method responsible for checking low stock levels was then executed.

The expected result was for the system to return `true`, indicating that the stock level was critically low. However, the method returned `false`, causing the test to fail.This indicates a **logic issue in the implementation** of the low-stock detection mechanism, meaning that the system does not correctly notify Managers when an item's stock falls below the predefined threshold.

REQ-14

The system shall allow Managers to restock items.

For this requirement, an item with very low stock was restocked using the ManagerController. The test verified that the item's quantity increased correctly and that the item was no longer considered out of stock.

The test was successful, confirming that the system allows Managers to restock items.

REQ-15

The system shall allow Managers to view sales summaries for a selected period (daily or monthly).

This requirement was tested using the Inventory class, which aggregates sales data and provides reports for specific time periods. A report was generated for a defined time interval and validated to ensure that the returned value was non-negative and consistent.

The test was successful, confirming that the system can generate sales summaries for a selected period.

REQ-16

The system shall allow Administrators to create, update, and delete employee accounts.

This requirement was tested through the AdministratorDashboardController and AdministratorDashboardView by simulating real user interactions such as selecting employees from the UI list and triggering button actions.

However, this requirement could not be fully executed during system testing. The AdministratorDashboardController relies on employee data being loaded from persistent storage (files/DAO). Since the persistence layer is not initialized in the testing environment, the UI lists cannot be populated correctly and employee removal actions cannot be completed.

This limitation is caused by missing system infrastructure rather than incorrect system behavior. As a result, this requirement could not be fully validated during system testing.